

Linux Embebido

Anexo

Embedded Linux

Annex

Autor: Ing. Marín Ariel (Adjunto)

Auxiliares Becarios

Fabrizio Carlassara (BIDOC)

Pablo Gómez (BIDOC)

Alexis de la Cruz Hernandez (BIS)

Resumen

Este anexo sobre Linux Embebido tiene como proposito indicar las relaciones entre cada uno de los archivos que se utilizan, entre ellos tenemos el Device Tree, los modulos de kernel y aplicación de usuario.

Palabras clave: Linux embebido; Archivos; Kernel

Abstract

This annex on Embedded Linux aims to indicate the relationships among the various files used, including the Device Tree, kernel modules, and the user application.

Keywords: Embedded Linux; Files; Kernel

1. Interacción de dispositivos en Linux

1.1. Espacio de usuario vs. Espacio de Kernel:

El *espacio de usuario* es donde se ejecutan las aplicaciones regulares de forma restringida. El *espacio de kernel* es la zona privilegiada donde reside el núcleo del SO y los controladores (drivers), teniendo acceso directo al hardware. Para que una aplicación acceda al hardware, debe cruzar esta frontera mediante "llamadas al sistema"(syscalls).

1.2. Device Tree Overlay (. dtbo):

Mientras que el Device Tree base describe la placa principal estáticamente, los *overlays* permiten modificar esa topología de hardware en tiempo de ejecución. Son fundamentales ya que permiten agregar o reconfigurar nodos sin necesidad de recompilar o reiniciar el sistema.

1.3. Módulos de Kernel (. ko):

Son fragmentos de código, que pueden ser controladores de dispositivos, que se pueden cargar (`insmod`) o descargar (`rmmmod`) dinámicamente en el kernel mientras el sistema está funcionando. Esto evita tener un kernel monolítico gigante y permite cargar solo lo que el hardware actual necesita.

1.4. Char Device y la App de Usuario:

Un *char device* es una abstracción que expone el hardware como un archivo (por ejemplo, `/dev/mi_periferico`). La **aplicación de espacio de usuario** interactúa con él abriéndolo y usando la API estándar de archivos. A través de la estructura `file_operations`, el kernel mapea estas llamadas al módulo: `open()` inicializa la conexión, `read()` permite a la app leer el estado (ej. valor de un botón), y `write()` permite enviar datos (ej. prender un LED).

1.5. IOCTL (Input/Output Control):

IOCTL es una llamada al sistema que permite a una aplicación controlar/comunicarse con char devices por fuera del flujo estándar de lectura/escritura de datos.

2. Diagrama desde espacio de usuario hasta espacio de kernel

El siguiente diagrama ilustra cómo se relacionan estos componentes cuando la aplicación de usuario interactúa (`read`, `write`, o `ioctl`) con el dispositivo de hardware subyacente.

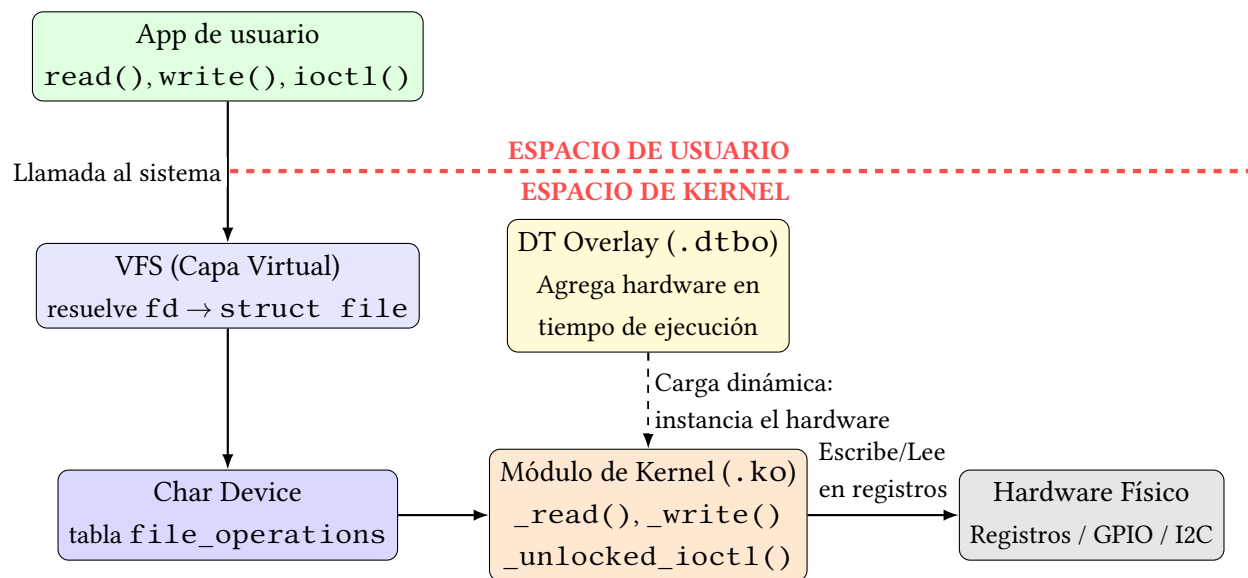


Figura 1: Interacción completa de los componentes arquitectónicos de Linux.